

Úvod do Reactu

Základy webového vývoje

Vaši vyučující

- Lukáš Grolig
- Petr Wehrenberg
- Filip Kaštovský
- Tomáš Sedláček
- Luděk Bártek

A samozřejmě skvělý tým cvičících

O kurzu

- naučíme vás tvořit webové aplikace
- procházíme znalosti do šířky - cílem je umět doručit projekt
- webové technologie se často mění, takže indepth znalosti by za rok zastaraly

Motivace

Chceme vás naučit tvořit aplikace tak, aby jste po tomto kurzu získali práci developera.

Co se naučíme

1. Informace o předmětu, Node.js, React (Lukáš Grolig)
2. React komponenty, úvod do CSS, UI (Petr Wehrenberg)
3. Úvod do Typescriptu (Filip Kaštovský)
4. REST API, dotazování, validace dat (Lukáš Grolig)
5. Databáze - modelování, ORM, migrace, seedování (Tomáš Sedláček)
6. Routování - Tanstack Router, Tanstack Start (Lukáš Grolig)
7. React - formuláře, state management (Petr Wehrenberg)
8. AI Development, Agentic Development (Petr Wehrenberg)
9. OAuth, RBAC, AuthZ, užitečné služby, telemetrie (Filip Kaštovský)
10. Pipelines, Deployment (Lukáš Grolig)
11. Zvaná přednáška - Vertice (Petr Krontorád, Martin Macák, Jan Hrdina)
12. XML - schéma, validace, API pro zpracování XML (L. Bártek)
13. XPath, XML Schema, API pro zpracování XML, XML - transformace (L. Bártek)

Doporučené IDE

- VS Code/Cursor
- Webstorm

Podmínky úspěšného ukončení

- 70 bodů projekt
- 30 bodů zkouška (odpovědník)

Cvičení

- povinná účast
- možnost 4 neomluvených absencí

Projekt

- hlavní součást hodnocení
- kombinuje znalosti jiných předmětů, které jste do teď absolvovali
- team lead se přihlásí na projekt a sestaví tým
- tým navrhne funkcionalitu a rozpadne ji
- odhadne náročnost částí a přiřadí body každé části
- na milestonu dostanete body, které si potom podle toho přepočítáte

Projekt

- 3 milestony
- závěrečná obhajoba

AI a motivace

Week 01 - Introduction to Node.js ecosystem, NPM and React

Lukáš Grolig et al.

Node.js

- An asynchronous event-driven JavaScript runtime
- Allows JS to execute outside a browser environment
- Designed to build scalable network applications
- Built on Chrome's [V8 JavaScript Engine](#)

Good Node.js use-cases

Use-case	Main benefits
Applications with heavy I/O requests (working with the database, filesystem...)	Using asynchronous calls and event loop instead of processes
Real-time apps (collaborative apps, chat...)	Great ability to handle multiple real-time connections at once
A stateless, client-server communication (RESTful APIs)	Non-blocking asynchronous processing of requests

NPM & Corepack

- Default **Node Package Manager**
- Libraries are stored in currently the world's largest software registry `npmjs.org`
- Corepack is an experimental feature containing other package managers: `Yarn`,
`pnpm`

```
corepack enable # Not necessary from Node 18+  
yarn # is now available  
pnpm # is now available
```

New alternative: bun

Initializing an NPM project

```
npm init  
# bun alternative:  
bun init
```

- Creates **package.json** file, which contains:
 - Project description
 - Dependencies
 - Runnable commands

Development packages (f.e. types, configurations, debugging packages)

```
npm install --save-dev typescript
```

```
# alternatively, preferred way:  
npm i -S -D typescript
```

```
# bun alternative:  
bun add --dev typescript
```

Regular dependencies

```
npm install --save react react-dom
```

```
# alternatively, preferred way:  
npm i -S react react-dom
```

```
# bun alternative:  
bun add react react-dom
```

These commands:

- Add entries to a `package.json` file

Global dependencies

- Some dependencies can be installed globally in the `node` installation folder
- A global dependency is available for all Node projects on the machine
- Less space is required because the dependency is installed only once

```
npm install --global --save typescript
```

```
# or
```

```
npm i -S -g typescript
```

```
# bun alternative:
```

```
bun add --global typescript
```

- Main disadvantage: dependency is not tracked in our `package.json` file
- Other developers **will not be able to know** which dependencies need to be installed in order to work on the project
- Usually **NOT** recommended for the reasons above

package.json file

```
{
  "name": "my_package",
  "description": "",
  "version": "1.0.0",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "https://github.com/monatheoctocat/my_package.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "bugs": {
    "url": "https://github.com/monatheoctocat/my_package/issues"
  },
  "homepage": "https://github.com/monatheoctocat/my_package"
}
```

.lock files

- Contains locked dependency versions resolved from package versioning
- Contains order of dependency installs (different order can cause issues)
- Must be committed to source control

Package versioning: caret and tilde

- Packages have different versions: npm packages (should) use **semantic versioning** (SemVer)
- Semantic versioning reflects the **state of the API** of the package
- Following these principles, published packages have three tiers of versions:

`Major.Minor.Patch`

Package versioning: caret and tilde

Major updates

- Backwards **compatibility is NOT guaranteed**
- This means the software can have a completely different API (function names, features, methodology, principles) between *any* two major versions

Minor updates

- Backwards **compatibility IS guaranteed**
- We can **add** new features and **deprecate** old ones (discourage people from using the older features while keeping them for compatibility's sake)

Patch updates

- Used for internal changes: bugfixes, optimizations, refactoring

Package versioning: caret and tilde

package.json:

```
{
  "name": "my-project",
  "version": "1.0",
  "dependencies": {
    "useLatestVersion": "*", // use whatever is the latest version
    "matchMajorVersion": "^1.4.3", // use any (usually latest) major 1.x.x version
    "matchMinorVersion": "~4.2.1", // use any (usually latest) minor 4.2.x version
    "matchExactVersion": "3.4.1" // use exactly the specified version
  }
  // rest of the package.json...
}
```


Motivation

- Most of the time, there is a need to update the content of a website dynamically
- Static HTML pages are unusable here, as you would need to manually update the pages
- To create a dynamic website, you could use a library like jQuery
- Nowadays, jQuery is still used on legacy systems but has mostly been replaced

What is SPA?

- SPA = Single Page Application
- The app is running on the client.
- Applications are reactive and responsive to changes on the server.
- Applications are communicating with the server to show the latest information or perform some state change.
- The same app can be used anywhere.
- Takes more time to develop.

Single Page Application	Server Side Application
can be written in JS or language targeting JS or WASM low consumption of server resources more performance on client required PWA is possible = apps can work without connection no requirement for refresh upon change	can be written in any lang client receives only HTML server resources heavy not many requirements on client application is fast to develop no offline application (PWA) every action needs reload

Vite: A Modern Build Tool for Frontend Development

Why Vite?

- **Faster Development:** Instant server startup & hot module replacement (HMR)
- **ES Module-Based:** Uses native ES modules instead of bundling everything upfront
- **Optimized for Modern Frameworks:** Supports React, Vue, Svelte, and more
- **Lightning-Fast Builds:** Uses Rollup for optimized production builds

Getting Started with Vite

```
bun create vite@latest my-app  
cd my-app  
bun install  
bun run dev
```

React.js

- JavaScript library for building user interfaces.
- Created to handle large-scale applications like Facebook, Messenger, Instagram, Airbnb, Netflix, and others. Used by major companies, including Microsoft.
- They do a really good job delivering backward compatibility.
- React is only concerned with state management and rendering that state to the DOM, most of the time additional libraries must be used
- Uses Client Side Rendering - rendering pages directly in the browser using JavaScript
- All logic, data fetching, templating and routing are handled on the client rather than the server.

Javascript syntax extension (JSX) / (TSX)

```
const element = <p>You clicked 0 times</p>;
```

- JSX is also called Javascript XML and is used to create React elements
- Any JS expression can be put into JSX and is an expression itself
- Using `class` is not permitted inside JSX, instead `className` is must be used

```
const element = <p className="clicksNumber">You clicked {count} times</p>;
```

Under the hood, JSX compiles to `React.createElement()` calls, but you rarely need to think about this.

Fragments

- Components must return a single root element
- Use fragments (`<>...</>`) to group elements without adding extra DOM nodes

```
const UserInfo = ({ name, email }) => {  
  return (  
    <>  
      <h2>{name}</h2>  
      <p>{email}</p>  
    </>  
  );  
};
```

Components

- Components let you split the UI into independent, reusable pieces, and think about each piece in isolation.
- Components are not limited to React, pure JS, and every other framework that uses them.
- For example, implementation of Components in pure JS can be done using [Web Components](#)
- Conceptually, components are like JavaScript functions. They accept arbitrary inputs (called "props") and return elements describing what should appear on the screen.

```
export const Counter = ({ count }: { count: number }) => {  
  return (  
    <div className="counter">  
      <p>You clicked {count} times</p>  
    </div>  
  );  
}
```

```
// Rendering our component  
<Counter count={5} />
```

Components in React

- The first letter of React component must be capitalized, otherwise React will recognize it as an HTML tag
- When React sees an element representing a user-defined component, it passes JSX attributes and children to this component as a single object. We call this object "props".
- Props are Read-Only, therefore a component must never modify its props.
- While using TypeScript it is best practice to define an interface containing props

```
interface CounterProps {  
  count: number;  
}  
  
export function Counter(props: CounterProps) {}
```

Components in React

- Functions and event handlers can be defined inside a component

```
export const Counter = () => {  
  const [count, setCount] = useState(0);  
  
  const incrementCount = () => {  
    setCount(count + 1);  
  }  
  
  return (  
    <div className="counter">  
      <p>You clicked {count} times</p>  
      <button onClick={incrementCount}>  
        Increment count  
      </button>  
    </div>  
  );  
}
```

Component Lifecycle

- Every component has a life cycle, which consists of 3 phases
 - First is mounting, which is when the component is rendered for the first time
 - Second is updating, which is every subsequent render
 - Third and last is unmounting, which is when we remove the component from the DOM
- The Effect hook is called in all 3 phases, but we will talk about it later in the lecture

Conditional render

```
export const Counter = ({count}) => {  
  if (count > 0) {  
    return <p>You clicked {count} times</p>;  
  }  
  return <p>You didn't click yet.</p>;  
}
```

How to prevent component render

```
export const WarningBanner = ({ warn }: { warn: boolean }) => {  
  if (!warn) {  
    return null;  
  }  
  
  return <div className="warning">Warning!</div>;  
};
```


React hooks

- Hooks let you use more of React's features without classes.
- It allows you to use state and other React features without writing a class. Hooks are the functions that "hook into" React state and lifecycle features from function components.
- Calling Hooks inside loops, conditions, or nested functions is not permitted.
- Instead, call hooks only from React function components or from custom hooks

For example:

- **State Hook**
- **Effect Hook**
- Context Hook
- Reducer Hook
- Layout effect hook

State hook

```
const [state, setState] = useState(initialState);
```

- Initially, the value of the state is set to initialState
- Value of the state is changed with setState
- setState not only changes the value but tells each component that uses the given state to rerender

- For example: Creates a button and a counter which increments with each click
- If normal variables were used, the value would change, but the component wouldn't rerender

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <p>You clicked {count} times</p>  
      <button onClick={() => setCount(count + 1)}>Click me</button>  
    </div>  
  );  
}
```

Effect hook

```
useEffect(func);
```

- Adds an ability to perform side effects from a function component
- Calls passed function after every render
- Multiple effect hooks can be placed inside one component

```
useEffect(func, [state]);
```

- The Effect hook can have a second parameter - a **dependency array**
- The function is called only when the specified dependencies change
- For example: The given function will be called only after `count` changes

```
useEffect(() => {  
  document.title = `You clicked ${count} times`;  
}, [count]);
```

- With an **empty** dependency array `[]`, the effect runs only once on mount

```
useEffect(() => {  
  console.log("Component mounted");  
}, []);
```

Lists

- In React transforming arrays into lists of elements is nearly identical.
- You can build collections of elements and include them in JSX using curly braces {}.

```
const numbers = [1, 2, 3, 4, 5];  
const listItems = numbers.map((number) => <li>{number}</li>);  
  
// later in the code  
<ul>{listItems}</ul>;
```

- This example is bad because it doesn't use `key`, more on the next slide
- You can also put this code into a component and pass the numbers as props

Keys

- In React, list keys help to identify which items have changed, are added, or are removed
- The best way to pick a key is to use a string that uniquely identifies a list item among its siblings. For example ID from data.
- Item index can be also used as a key, but only as a last resort
- Use index as a key only if these conditions are met, otherwise it may break your application and display the wrong data
 - the list and items are static
 - the items in the list have no ids
 - the list is never reordered or filtered

Virtual DOM (VDOM)

- React keeps a lightweight "virtual" representation of the UI in memory
- When state changes, React computes a new virtual tree and compares it with the previous one (**reconciliation**)
- Only the differences are applied to the real DOM - this minimizes expensive DOM operations
- This is why `key` props matter in lists - they help React identify which elements changed

Hooks and objects

- Sometimes we need to store an object as a state instead of a primitive type
- Mutating object properties directly does **not** trigger a rerender
- Use the spread operator `...` to create a new object reference

```
const [user, setUser] = useState({ name: "Alice", age: 25 });
```

```
// Wrong – mutating the existing object, no rerender
```

```
user.name = "Bob";
```

```
setUser(user);
```

```
// Correct – creating a new object
```

```
setUser({ ...user, name: "Bob" });
```

Shipping the react app

- Compile, minify typescript
- Process, bundle css
- Copy assets
- Now your app can be served by any conventional file server (nginx, caddy, apache)

Importing a CSS stylesheet

```
import './Styles.css';
```

- Css modules - create a special css file with `.module.css` extension

```
import * as styles from './counter.module.css';  
<p className={styles.count}>You clicked {count} times</p>;
```

- All modules are locally scoped, therefore you don't have to worry about name collision

React ecosystem - meta-frameworks

- **Next.js** - the most popular React framework, supports SSR, static generation, and more
- **Remix** - focused on web standards, server rendering, and progressive enhancement
- **Tanstack Start** - lightweight, type-safe, built on top of Tanstack Router
- In this course we will use **Tanstack Router / Tanstack Start** for routing and SSR

Simple applications

- start with Pico CSS

Basic layout - container

```
<body>  
  <header>...</header>  
  <main>...</main>  
  <footer>...</footer>  
</body>  
  
<div class="container"></div>
```

Grid

```
<div class="grid">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
  <div>4</div>  
</div>
```


Form

```
<form>
  <fieldset role="group">
    <input name="email" type="email" placeholder="Enter your email" autocomplete="email" />
    <input type="submit" value="Subscribe" />
  </fieldset>
</form>
```